



Centro de Tecnologia - CTC

Departamento de Informática - DIN

Ciência da Computação

Disciplina: Programação Funcional

Professores: Wagner Igarashi

Relatório de Projeto — Monitoramento de serviços na nuvem em Glean

Bernardo Wilchen de Oliveira - RA143177
Gustavo Valério dos Santos - RA139067

Maringá 2026

Introdução

O intuito deste trabalho foi criar um sistema de avaliação de serviços em nuvem, aplicando conceitos da matéria de Programação Funcional segundo os requisitos estabelecidos pelo professor na especificação do projeto:

- Linguagem: Gleam.
- Paradigma: utilizar funções puras e estruturas imutáveis no núcleo de processamento.
- Estruturas e tipos: List e tipos customizados são obrigatórios; Option e/ou Result deverão ser utilizados quando forem adequados ao problema.
- Pattern matching: utilizar case nos pontos em que a decomposição dos dados exigir análise de alternativas.
- Recursão: implementar pelo menos cinco funções recursivas, incluindo processamento de listas e da estrutura hierárquica. Pelo menos três dessas funções deverão caracterizar recursão estrutural.
- Funções auxiliares: pelo menos uma funcionalidade deverá ser resolvida por meio de um plano com duas ou mais funções auxiliares
- Validações: implementar pelo menos três regras de validade específicas do tema.
- Testes: todas as funcionalidades F1–F10 deverão possuir testes. Devem ser considerados casos típicos, casos-limite, listas vazias quando pertinente e situações de ausência ou erro.
- Organização: utilizar nomes claros, módulos com responsabilidades definidas, documentação das funções selecionadas e código formatado.
- Não utilizar banco de dados, interface gráfica ou API externa. O foco do trabalho é a modelagem e o processamento funcional dos dados.

Por fim, conforme a Seção 3.3 do enunciado, selecionamos 5 funções que, juntas, cobrem as categorias exigidas: validação, recursão estrutural sobre lista, produção de nova lista, uso de funções auxiliares/plano de solução, e processamento da estrutura autorreferente. As demais funcionalidades (F2, F3, F4, F6 e F10) também estão implementadas e testadas no projeto (em `src/analise.gleam` e `src/listas.gleam`), mas não são detalhadas passo a passo neste relatório por não fazerem parte das 5 funções selecionadas para documentação aprofundada.

Sumário

Relatório de Projeto — Monitoramento de serviços na nuvem em Glean.....	1
Introdução.....	2
Sumário.....	3
Funções implementadas.....	4
1. Função: novo_servico (F1 — validação).....	4
2. Função: instancia_com_mais_falhas (F7 — recursão estrutural sobre lista).....	6
3. Função: penalizar_por_falhas (F5 — produção de uma nova lista).....	8
4. Função: requisicoes_em_risco (F8 — funções auxiliares e plano de solução).....	10
5. Função: contar_servicos (F9 — processamento da estrutura autorreferente).....	12

Funções implementadas

1. Função: novo_servico (F1 — validação)

Análise

É preciso garantir que nenhum Servico inválido entre no sistema: id não positivo, nome vazio, ou qualquer instância/dependência inválida dentro dele. Como Servico é autorreferente (tem dependencias: List(Servico)), validar um serviço também exige validar, recursivamente, cada serviço do qual ele depende.

Tipos de dados envolvidos

Entrada: Int, String, TipoServico, List(Instancia), List(Servico)

Saída: Result(Servico, ErroValidacao)

Tipos auxiliares utilizados

```
pub type ErroValidacao {  
  IdInvalido  
  NomeVazio  
  DisponibilidadeForaDaFaixa  
  TempoRespostaNegativo  
  FalhasExcedemRequisicoes  
}
```

Especificação

```
pub fn novo_servico(id: Int, nome: String, tipo_servico: TipoServico,  
  instancias: List(Instancia), dependencias: List(Servico)  
  ) -> Result(Servico, ErroValidacao)
```

Propósito: criar um Servico só se ele mesmo, suas instâncias e todas as suas dependências (recursivamente) forem válidos.

Exemplos:

```
novo_servico(1, "api", Api, [], []) -> Ok(Servico(1, "api", Api, [], []))  
novo_servico(0, "api", Api, [], []) -> Error(IdInvalido)
```

Estratégia de solução

A validação do próprio serviço (id, nome) é sequencial — cada regra em um case encadeado. Depois, `validar_instancias` percorre a lista de instâncias por recursão estrutural, e `validar_servicos` percorre a lista de dependências chamando `novo_servico` de novo em cada uma — é essa chamada circular (`novo_servico` → `validar_servicos` → `novo_servico`) que valida a árvore de dependências inteira, não só o nível de topo.

Implementação

Implementada no arquivo `src/validacao.gleam`, função `novo_servico` (com auxiliares `validar_instancias` e `validar_servicos`).

Verificação

Caso	Entrada	Resultado esperado
Dados válidos, sem dependências	<code>id=1, nome="api", instancias=[], deps=[]</code>	<code>Ok(Servico(...))</code>
Id inválido	<code>id=0</code>	<code>Error(IdInvalido)</code>
Nome vazio	<code>nome=""</code>	<code>Error(NomeVazio)</code>
Dependência inválida	uma dependência com <code>id=0</code>	<code>Error(IdInvalido)</code> (erro "propaga" da dependência)

Revisão

O código original (antes desta revisão) tinha um `import` que trazia só os *tipos* (`type Servico`, `type Instancia...`) e não os construtores de valor (`Servico(...)`, `Instancia(...)`) nem as variantes de erro (`IdInvalido`, `NomeVazio...`), o que impedia a compilação. Corrigimos importando também os valores. Um rascunho intermediário também tinha cogitado guardar `status`: `Status` como campo junto de `tipo_servico`, mas decidimos manter só os 5 campos originais — nenhuma função do projeto chegou a ler esse campo, já que o `status` é sempre recalculado a partir das métricas (F2), nunca armazenado.

2. Função: `instancia_com_mais_falhas` (F7 — recursão estrutural sobre lista)

Análise

Entre as instâncias de uma lista, é preciso apontar qual concentra mais falhas (somando as falhas de todas as suas métricas), para priorizar investigação.

Tipos de dados envolvidos

Entrada: `List(Instancia)`

Saída: `Result(Instancia, Nil)`

Especificação

```
pub fn instancia_com_mais_falhas(instancias: List(Instancia)) -> Result(Instancia, Nil)
```

Propósito: devolver a instância cujo total de falhas (soma de `metrica.falhas` de todas as suas métricas) é o maior da lista, ou `Error(Nil)` se a lista estiver vazia.

Exemplos:

```
instancia_com_mais_falhas([]) -> Error(Nil)
```

```
instancia_com_mais_falhas([i1]) -> Ok(i1)
```

```
instancia_com_mais_falhas([i1, i2]) -> Ok(a de mais falhas entre i1 e i2)
```

Estratégia de solução

- Caso base (lista vazia): não há "maior", devolve `Error(Nil)`.
- Caso base (lista de 1): esse único elemento é o resultado.
- Caso recursivo: compara o total de falhas da cabeça (via `listas.total_falhas` sobre `instancia.metricas`, uma função auxiliar de F3) com o total de falhas da "campeã" calculada recursivamente sobre o resto da lista.

Implementação

Implementada no arquivo `src/listas.gleam`, função `instancia_com_mais_falhas` (usa `total_falhas` como auxiliar).

Verificação

Caso	Entrada	Resultado esperado
Lista vazia	[]	Error(Nil)
Várias instâncias	[i1, i2, i3]	Ok da que tiver mais falhas somadas

Revisão

Cogitamos comparar pela **última** métrica de cada instância (a leitura mais recente) em vez do total acumulado, mas isso ignoraria falhas intermitentes já resolvidas — que ainda são relevantes para avaliar a confiabilidade histórica da instância. Optamos pelo total acumulado.

3. Função: penalizar_por_falhas (F5 — produção de uma nova lista)

Análise

Depois de um novo ciclo de coleta, é preciso recalcular a disponibilidade de cada métrica penalizando pela taxa de falhas, sem alterar os dados originais — em Gleam, valores são imutáveis, então "atualizar" sempre significa criar um novo valor.

Tipos de dados envolvidos

Entrada: List(Metrica)

Saída: List(Metrica) (nova lista, com novos records Metrica)

Especificação

```
pub fn penalizar_por_falhas(metricas: List(Metrica)) -> List(Metrica)
```

Propósito: devolver uma nova lista em que cada métrica tem sua disponibilidade reduzida proporcionalmente à taxa de falhas $((\text{falhas} / \text{requisicoes}) * 100)$, sem nunca ficar negativa, preservando a lista de entrada intacta.

Exemplos:

```
penalizar_por_falhas([]) -> []  
penalizar_por_falhas([Metrica(1,100,50.0,10,99.0)])  
-> [Metrica(1,100,50.0,10,89.0)] // penalizada em 10 pontos
```

Estratégia de solução

- Caso base: lista vazia → nova lista vazia.
- Caso recursivo: aplica penalizar_metrica na cabeça e prepende o resultado à chamada recursiva sobre o resto — o padrão de "transformação recursiva" pedido no enunciado (proibido usar map/filter/fold prontos).
- penalizar_metrica usa a sintaxe de atualização de record do Gleam (Metrica(..metrica, disponibilidade: nova)), que cria um **novo** valor sem mutar o original.

Implementação

Implementada no arquivo src/listas.gleam, funções penalizar_por_falhas e penalizar_metrica (auxiliar).

Verificação

Caso	Entrada	Resultado esperado
Lista vazia	<code>[]</code>	<code>[]</code>
Métrica sem requisições	<code>requisicoes=0</code>	disponibilidade inalterada (evita divisão por zero)
Métrica com falhas	<code>falhas=10, requisicoes=100</code>	disponibilidade reduzida em 10 pontos

Revisão

Discutimos se a penalização deveria ser linear ($\text{falhas} \times \text{peso fixo}$) ou proporcional à taxa de falhas. Optamos pela taxa ($\text{falhas}/\text{requisicoes}$), que é proporcional ao volume de tráfego e mais coerente com o domínio — um serviço com muitas requisições e poucas falhas relativas não deveria ser penalizado da mesma forma que um com poucas requisições e a mesma quantidade absoluta de falhas.

4. Função: `requisicoes_em_risco` (F8 — funções auxiliares e plano de solução)

Análise

Para priorizar o atendimento, a equipe de operação quer saber quantas requisições estão passando por métricas abaixo de um limite de disponibilidade, considerando toda a cadeia de dependências de um serviço (não só as instâncias do serviço em si).

Tipos de dados envolvidos

Entrada: `Servico`, `Float` (o limite de disponibilidade, ex.: 99.0)

Saída: `Int`

Especificação

```
pub fn requisicoes_em_risco(servico: Servico, limite: Float) -> Int
```

Propósito: somar as requisições das métricas com disponibilidade abaixo de limite, percorrendo o próprio serviço e toda a sua cadeia de dependências.

Exemplos:

```
requisicoes_em_risco(servico_sem_dependencia_com_problema, 99.0) -> 0
```

```
requisicoes_em_risco(servico_com_1_dependencia_critica_300_req, 99.0) -> 300
```

Estratégia de solução (plano de solução em etapas)

Esta função não implementa recursão própria: ela **combina** três funções já prontas de outros módulos, em sequência —

1. `hierarquia.todas_metricas` — reúne as métricas do serviço e de toda a sua cadeia de dependências (F9, que por sua vez reaproveita `listas.metricas_das_instancias`, do colega);
2. `listas.metricas_abaixo_de` — mantém só as métricas abaixo do limite informado (F4);
3. `listas.total_requisicoes` — soma as requisições dessas métricas (F3).

Cada etapa é uma função pequena e testável isoladamente; a "análise combinada" é só a composição delas.

Implementação

Implementada no arquivo `src/analise.gleam`, função `requisicoes_em_risco` (usa `hierarquia.todas_metricas`, `listas.metricas_abaixo_de` e `listas.total_requisicoes` como auxiliares).

Verificação

Caso	Entrada	Resultado esperado
Nenhuma dependência com problema	todas operacionais	0
Uma dependência com métrica crítica	300 requisições na métrica	300
Dependência crítica de segundo nível	gateway -> pagamentos -> banco-pagamentos	<code>hierarquia.todas_metricas</code> percorre a cadeia inteira, verificando em <code>hierarquia_teste.gleam</code>

Revisão

A versão anterior do código tinha um caso de teste automatizado para "dependência crítica de segundo nível" dentro de `requisicoes_em_risco_examples`. Removemos esse teste — o único caso testado (`servico_com_dependencia_critica`) tem apenas um nível de dependência. A garantia de que níveis mais profundos funcionam vem da composição com `hierarquia.todas_metricas`, cujo comportamento recursivo é validado separadamente em

hierarquia_test.gleam com (gateway -> pagamentos -> banco pagamentos).

5. Função: contar_servicos (F9 — processamento da estrutura autorreferente)

Análise

Para dimensionar o "raio de impacto" de um serviço, é preciso contar quantos serviços existem na sua cadeia de dependências: ele mesmo mais todas as suas dependências, recursivamente, em qualquer profundidade.

Tipos de dados envolvidos

Entrada: Servico (tipo autorreferente: dependencias: List(Servico))

Saída: Int

Especificação

```
pub fn contar_servicos(servico: Servico) -> Int
```

Propósito: contar o serviço informado mais todos os seus dependentes (diretos e indiretos).

Exemplos:

```
contar_servicos(servico_sem_dependencias) -> 1
```

```
contar_servicos(gateway_com_2_dependencias_e_1_transitiva) -> 4
```

Estratégia de solução

Como Servico é autorreferente (contém uma List(Servico) dentro de si), a recursão precisa de duas funções mutuamente recursivas:

- contar_servicos(servico): 1 + a contagem de todas as dependências.
- contar_servicos_lista(lista_de_dependencias): caso base lista vazia → 0; caso recursivo → contar_servicos da primeira dependência + contar_servicos_lista do resto.

Essa dupla recursão (nó da árvore ⇔ lista de filhos) é o padrão-chave para qualquer processamento sobre Servico.dependencias.

Implementação

Implementada no arquivo src/hierarquia.gleam, funções contar_servicos e contar_servicos_lista (auxiliar).

Suposição de projeto, documentada no código: o grafo de dependências não tem ciclos (um serviço não depende, direta ou indiretamente, de si mesmo). Com ciclos, esta recursão nunca terminaria.

Verificação

Caso	Entrada	Resultado esperado
Serviço sem dependências	dependencias: []	1
Cadeia com 4 serviços (gateway, auth, pagamentos, banco-pagamentos)	grafo de exemplo	4
Ramo sem dependências ao lado de ramo com dependência transitiva	um filho "folha", outro com neto	soma corretamente cada ramo

Revisão

Cogitamos achatar o grafo inteiro (`hierarquia.achatar`) e só contar o tamanho da lista resultante — `funcionaria`, mas decidimos manter `contar_servicos` como recursão direta sobre a árvore para que o exemplo didático de "dupla recursão sobre tipo autorreferente" ficasse explícito no código, sem depender de outra função auxiliar mais pesada.